

# World models: three paradigms

| Paradigm                     | What is generated                                    | Examples                                               | Use                         |
|------------------------------|------------------------------------------------------|--------------------------------------------------------|-----------------------------|
| 3D scene generation          | an explicit scene (Gaussian splats / NeRF) to render | Marble (World Labs, Nov 2025), DreamFusion, GSGen      | assets, simulation          |
| interactive video generation | the next frame given the user's action               | Genie 1 → 3 (DeepMind, 2024–Aug 2025), GameNGen, Oasis | playable worlds, robot data |
| latent world models          | the next latent state, not pixels                    | V-JEPA 2 (Meta, Jun 2025), PAN                         | planning, control           |

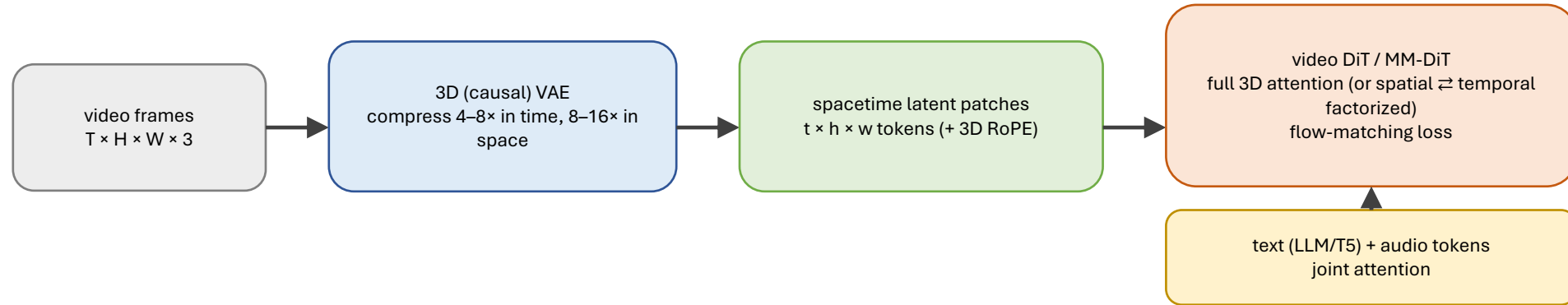
*Genie 3: real-time 720p interactive worlds lasting minutes, from a text prompt; limitations reported: short memory, latent actions, no explicit geometry*

A world model predicts what happens next given an action: generated 3D scenes, interactive video, or latent states

Genie (DeepMind): learns latent actions from unlabeled video; Genie 3 (Aug 2025) generates playable 720p worlds in real time

V-JEPA 2 (Meta, Jun 2025): non-generative — predicts masked latents, then plans with the predictor (ties to Lecture 5's JEPA)

# Video generators are latent flow-matching transformers with a time axis

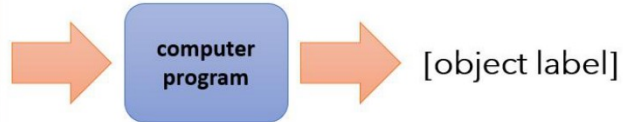


*Sora (Feb 2024), Movie Gen (Oct 2024, 30B), Wan 2.x, LTX-2, Veo 3 and Sora 2 (2025) all follow this template; older: 3D U-Nets with factorized attention*

A 3D VAE compresses time and space; spacetime patches become tokens with 3D positional encoding  
Full 3D attention (or spatial ⇔ temporal factorized attention in older 3D U-Nets); text and now audio tokens attend jointly with video tokens  
Sampling and RL tricks carry over from images: few-step distillation, CFG, reward fine-tuning

# What is reinforcement learning?

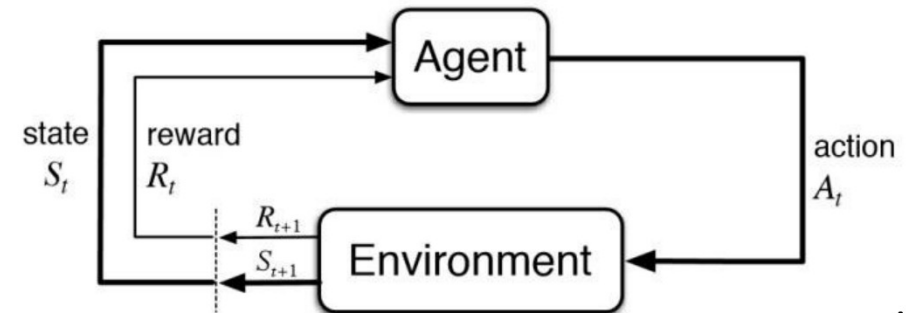
## supervised learning



input:  $\mathbf{x}$   
output:  $\mathbf{y}$   
data:  $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}$   
goal:  $f_{\theta}(\mathbf{x}_i) \approx \mathbf{y}_i$

← someone gives this to you

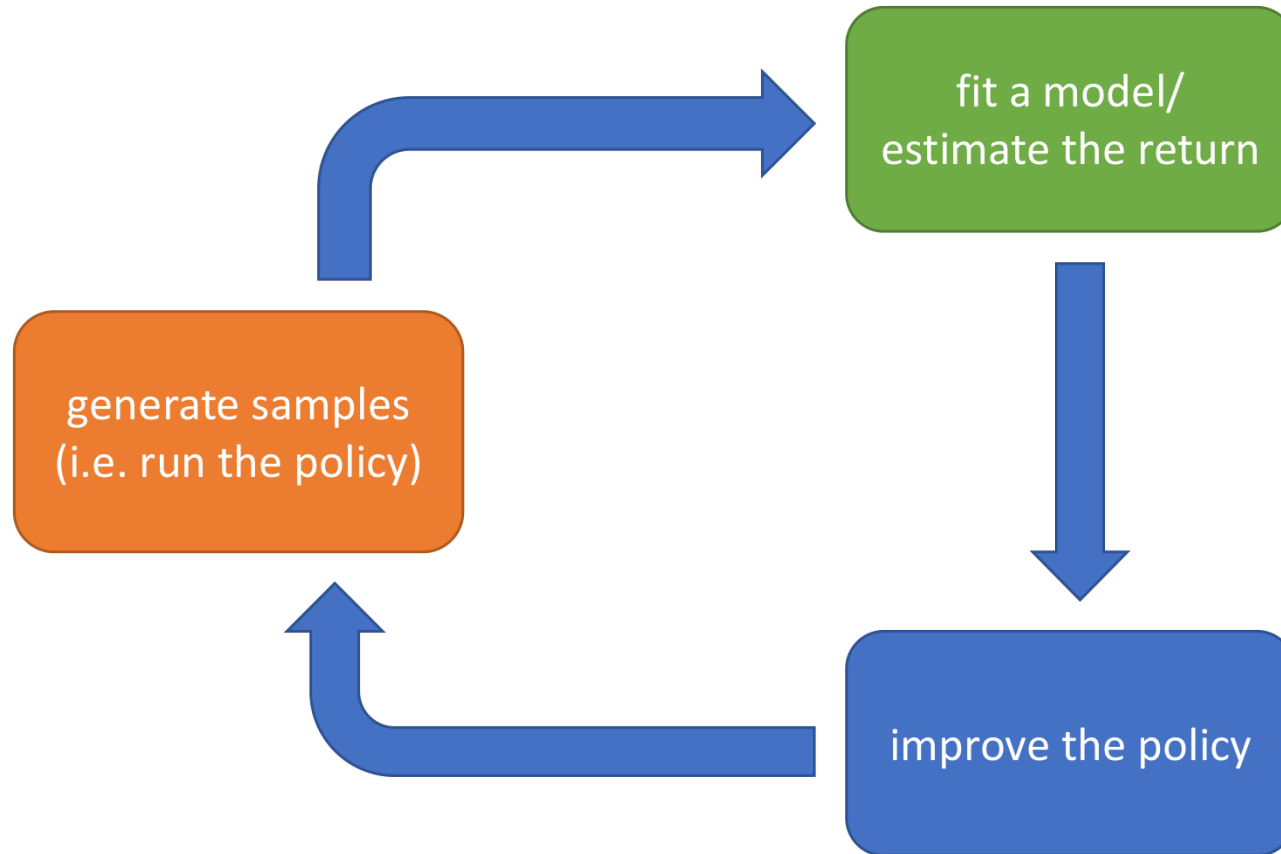
## reinforcement learning



input:  $\mathbf{s}_t$  at each time step  
output:  $\mathbf{a}_t$  at each time step  
data:  $(\mathbf{s}_1, \mathbf{a}_1, r_1, \dots, \mathbf{s}_T, \mathbf{a}_T, r_T)$   
goal: learn  $\pi_{\theta} : \mathbf{s}_t \rightarrow \mathbf{a}_t$   
to maximize  $\sum_t r_t$

pick your own actions

# The anatomy of a reinforcement learning algorithm

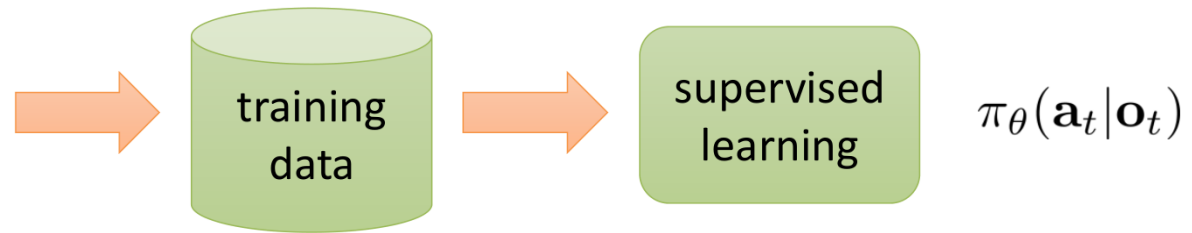
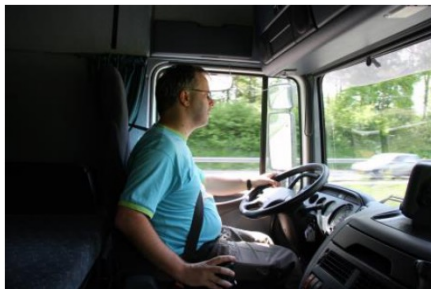
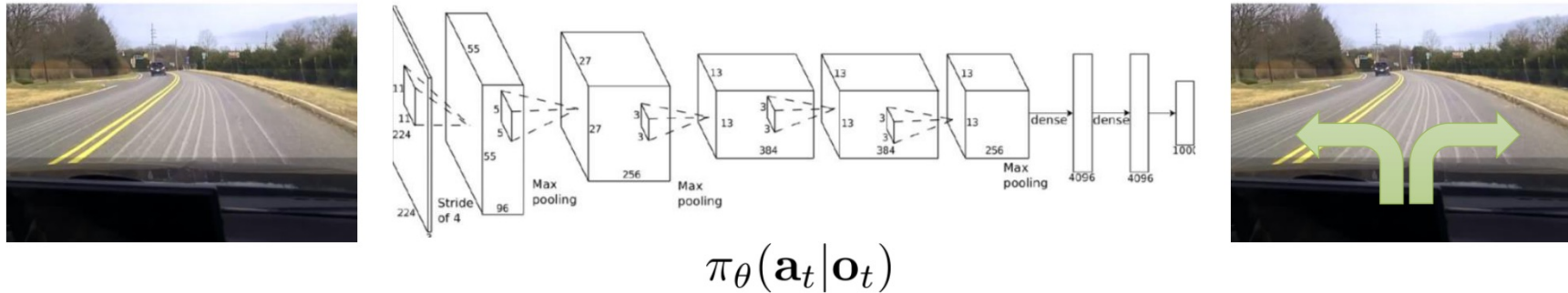


# Types of RL algorithms

$$\theta^* = \arg \max_{\theta} E_{\tau \sim p_{\theta}(\tau)} \left[ \sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

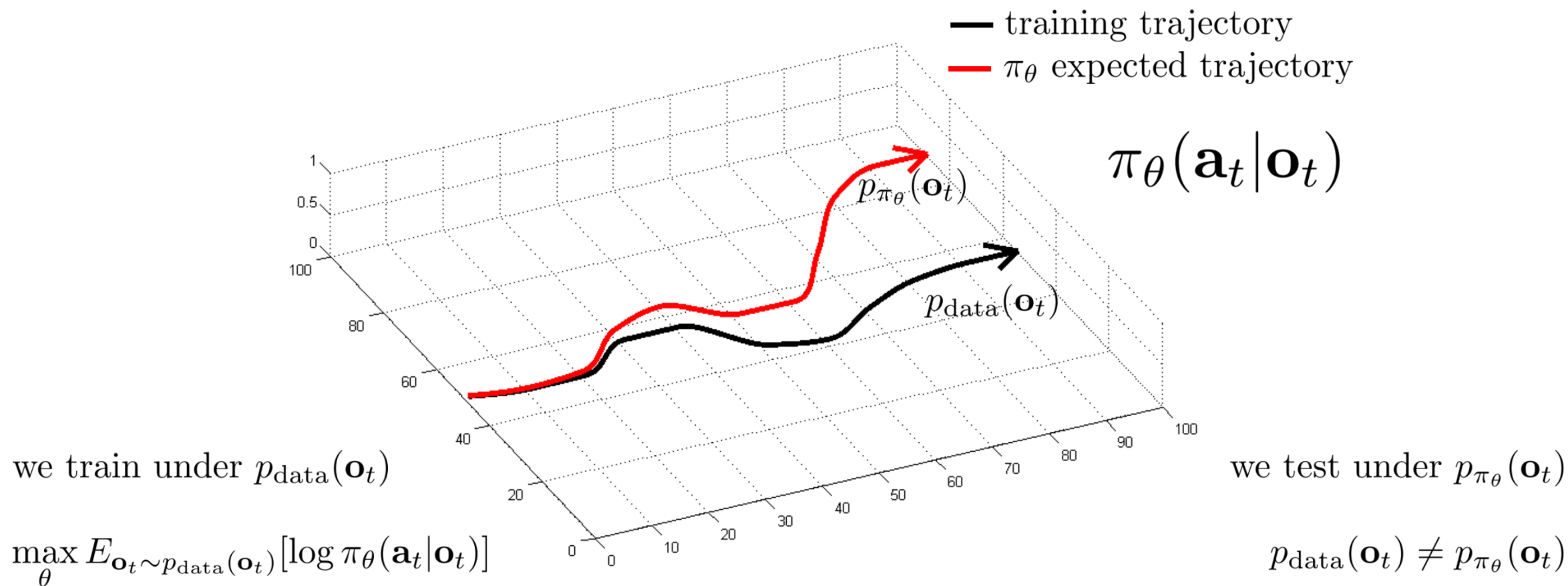
- Policy gradients: directly differentiate the above objective
- Value-based: estimate value function or Q-function of the optimal policy (no explicit policy)
- Actor-critic: estimate value function or Q-function of the current policy, use it to improve policy
- Model-based RL: estimate the transition model, and then...
  - Use it for planning (no explicit policy)
  - Use it to improve a policy
  - Something else

# Imitation Learning



behavioral cloning

# The distributional shift problem

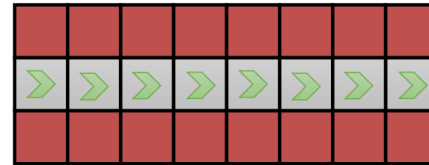
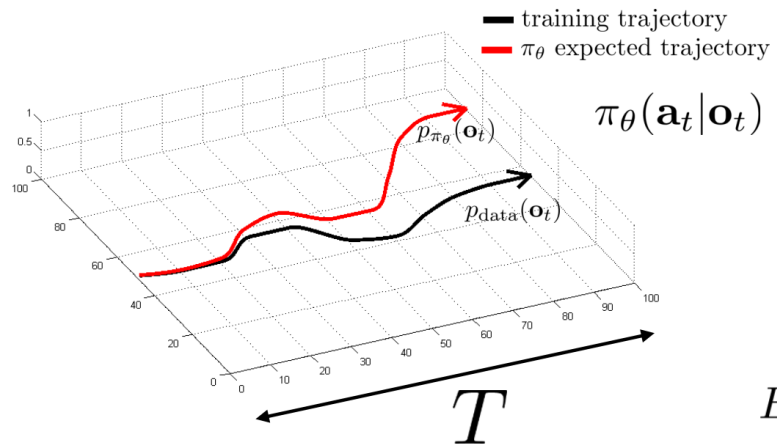


# Some analysis

$$c(\mathbf{s}, \mathbf{a}) = \begin{cases} 0 & \text{if } \mathbf{a} = \pi^*(\mathbf{s}) \\ 1 & \text{otherwise} \end{cases}$$

assume:  $\pi_\theta(\mathbf{a} \neq \pi^*(\mathbf{s}) | \mathbf{s}) \leq \epsilon$

for all  $\mathbf{s} \in \mathcal{D}_{\text{train}}$



$$E \left[ \sum_t c(\mathbf{s}_t, \mathbf{a}_t) \right] \leq \underbrace{\epsilon T}_{O(\epsilon T^2)} + \underbrace{\phantom{\epsilon T}}_{T \text{ terms, each } O(\epsilon T)}$$



# Can we make it work more often?

can we make  $p_{\text{data}}(\mathbf{o}_t) = p_{\pi_\theta}(\mathbf{o}_t)$ ?

idea: instead of being clever about  $p_{\pi_\theta}(\mathbf{o}_t)$ , be clever about  $p_{\text{data}}(\mathbf{o}_t)$ !

## DAgger: Dataset Aggregation

goal: collect training data from  $p_{\pi_\theta}(\mathbf{o}_t)$  instead of  $p_{\text{data}}(\mathbf{o}_t)$

how? just run  $\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$

but need labels  $\mathbf{a}_t$ !

- 
1. train  $\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$  from human data  $\mathcal{D} = \{\mathbf{o}_1, \mathbf{a}_1, \dots, \mathbf{o}_N, \mathbf{a}_N\}$
  2. run  $\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$  to get dataset  $\mathcal{D}_\pi = \{\mathbf{o}_1, \dots, \mathbf{o}_M\}$
  3. Ask human to label  $\mathcal{D}_\pi$  with actions  $\mathbf{a}_t$
  4. Aggregate:  $\mathcal{D} \leftarrow \mathcal{D} \cup \mathcal{D}_\pi$

# Robots and LLMs learn the same way in 2025–26

- **Vision-language-action models** = behavior cloning with a VLM backbone: RT-2 (Google DeepMind, 2023) emits actions as text tokens;  **$\pi 0$**  (Physical Intelligence, Oct 2024) adds a flow-matching action head (Lecture 10b);  **$\pi 0.5$**  (Apr 2025) targets open-world generalization; **Gemini Robotics** (Mar 2025)
- Then the same second stage as LLMs: RL fine-tuning on outcomes (success detectors) or human corrections (DAgger-style)
- LLMs: SFT (imitation)  $\rightarrow$  RLHF (learned reward)  $\rightarrow$  RLVR (verifier reward, Lecture 12); image/video generators: DDPO / Flow-GRPO (Lecture 10b)
- Levine's summary (Lecture 1): data teaches what the world looks like; optimization is what solves new problems